



Strongly Connected Components and Topological Sort.

1 Topological Sort

Topological sort of a directed graph is a ordering of its vertices such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering. The native application is in determining a compilation order. Imagine a huge source code, a compiler has to compile in such an order that when compiling a function, all the functions on which it depends have already been compiled.

Definition 1. (connectivity)

1. Let $G = (V, E)$ be an undirected graph. A **connected component** of G is a subset $U \subseteq V$ in which there exists a path between every two vertices, and U is 'maximal'¹ in the sense that any extension of it by a vertex results in two vertices that are not connected.
2. A non-directed graph G is said to be a **connected** graph if it only has one connected component.
3. Let $G = (V, E)$ be a directed graph. A **strongly connected component** of G is a subset $U \subseteq V$ in which for any pair of vertices $u, v \in U$, there exist both a directed path from u to v and a directed path from v to u , and U is also 'maximal' in the sense that any extension of it by a vertex results in two vertices that are not connected (from both sides).
4. A directed graph G is said to be a **strongly connected** if it only has one strongly connected component.

1.1 Depth First Search (DFS)

As its name implies, depth-first search searches "deeper" in the graph whenever possible. Depth-first search explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it. Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from

¹Local maximum.



Figure 5.1: Topological sort

which v was discovered. This process continues until all vertices that are reachable from the original source vertex have been discovered. If any undiscovered vertices remain, then depth-first search selects one of them as a new source, repeating the search from that source. The algorithm repeats this entire process until it has discovered every vertex.

```

1 DFS( $G$ ):
2   for  $v \in V$  do
3      $v.\text{visited} \leftarrow \text{False}$ 
4   end
5    $\text{time} \leftarrow 1$ 
6   for  $v \in V$  do
7     if not  $v.\text{visited}$  then
8        $\pi(v) \leftarrow \text{null}$ 
9       Explore( $G, v$ )
10    end
11  end

1 Explore( $G, v$ ):
2   Previsit( $v$ )
3   for  $(v, u) \in E$  do
4     if not  $u.\text{visited}$  then
5        $\pi(u) \leftarrow v$ 
6       Explore( $G, u$ )
7     end
8   end
9   Postvisit( $v$ )

1 Previsit( $v$ ):
2    $\text{pre}(v) \leftarrow \text{time}$ 
3    $\text{time} \leftarrow \text{time} + 1$ 

1 Postvisit( $v$ ):
2    $\text{post}(v) \leftarrow \text{time}$ 
3    $\text{time} \leftarrow \text{time} + 1$ 

```

Properties of depth-first search. Depth-first search yields valuable information about the structure of a graph. Perhaps the most basic property of depth-first search is that the predecessor subgraph G_π does indeed form a forest of trees since the structure of the depth-first trees exactly mirrors the structure of recursive calls

of explore-function. That is, $u = \pi(v)$ if and only if $\text{explore}(G, v)$ was called during a search of u 's adjacency list.

Additionally, vertex v is a descendant of vertex u in the depth-first forest if and only if v is discovered during the time in which $\text{pre}(u)$ has already been set but $\text{post}(u)$ hasn't². Another important property of depth-first search is that discovery and finish times have a parenthesis structure. If the explore procedure were to print a left parenthesis " $(u$ " when it discovers vertex u and to print a right parenthesis " $)u$ " when it finishes u , then the printed expression would be well-formed in the sense that the parentheses are properly nested.

The following theorem provides another way to characterize the parenthesis structure.

Theorem 1 (Parenthesis theorem). *In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:*

1. *the intervals $[\text{pre}(u), \text{post}(u)]$ and $[\text{pre}(v), \text{post}(v)]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest.*
2. *the interval $[\text{pre}(u), \text{post}(u)]$ is contained entirely within the interval $[\text{pre}(v), \text{post}(v)]$, and u is a descendant of v in a depth-first tree, or*
3. *the interval $[\text{pre}(v), \text{post}(v)]$ is contained entirely within the interval $[\text{pre}(u), \text{post}(u)]$, and v is a descendant of u in a depth-first tree.*

Let us use the following notation: we say that line (x) of the recursive call ' $\text{Explore}(G, v)$ ' is executed at time t if at time t the running of the algorithm got to line number x of Explore where the vertex argument is v . Notice that since Explore is called **only once** on each vertex, the notation is well defined.

Proof. It's enough to prove that for any pair of vertices v, u , the inequality $\text{pre}(v) < \text{pre}(u) < \text{post}(v)$ implies $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$.

Consider the call ' $\text{Explore}(G, v)$ '. Since the algorithm sets the value of $\text{post}(v)$ at line number (9), and it makes all of its recursive calls between lines (3)-(8), we have that the algorithm returns from its recursive calls before time $\text{post}(v)$. Furthermore, any line that has been executed during times $[\text{pre}(v), \text{post}(v)]$ is either a line of the call ' $\text{Explore}(G, v)$ ' or a line of one of the recursive calls.

In particular, we have that the line which sets $\text{pre}(u)$ is executed during times $[\text{pre}(v), \text{post}(v)]$, and therefore as part of a recursive call. In addition, because the recursive call was returned during the time interval, the assignment of $\text{post}(u)$ is made before $\text{post}(v)$. Thus, we have both $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$ and that u is a descendant of v in π ³.

□

Corollary 1. Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$.

²In terms of the presentation from last week, when u 's color is gray

³The algorithm defines π to be the recursive tree

2 DFS on DAGS

the pre is well calibrated on DAGS.

Claim 1. *Let $G = (V, E)$ be a graph, and let $v, u \in V$ such $u \rightarrow v$ (There is a path from u to v in G). And let π be a forest returned from the call **DFS**(G), If $\text{pre}(u) < \text{pre}(v)$ then v is a descendant of u in G .*

To prove Claim 1, we will define a graph G' that satisfies:

1. There is only single path between u to v in G' .
2. Let π' be the depth forest returned by the running of DFS over G' , then if v is a descendant of u in π' then is also a descendant of u in π .

Definition 2. Consider a graph G and vertices u, v such that there is a path from u to v . Let $e_1, e_2, \dots, e_m$ be the edges of the graph ordered by the access time of the DFS running over G . Let $G' \subset G$ be defined by taking the minimal prefix $e_1, e_2, \dots, e_i$ such that there is still a path from u to v . Let π' be the forest returned by the DFS running over G' , and notice that $\pi' \subset \pi$. Observe that by minimality, there is only one path between u and v in G' .

Proof. Consider vertices v, u such $u \rightarrow v$, in G , $\text{pre}(u) < \text{pre}(v)$ and G' as defined above. Assume without loss of generality that v is the first of such vertex on the path that is not a descendant of u in π' . Let v' be predecessor to v on that path, namely the last vertex on the ' $u \rightarrow v$ ' path before v .

Either $\text{pre}(v') < \text{pre}(u)$ or v' is a descendant of u . In the first case, we get an immediate contradiction to $\text{pre}(v) < \text{pre}(u)$. In the second case, we consider the recursive call '**Explore**(G', v')', and we have that $\text{pre}(v) < \text{pre}(v')$, or otherwise the algorithm will set $\pi'(v) \leftarrow v'$, and hence v is a descendant of u in π' .

So, let's focus on the case where $\text{pre}(v) < \text{pre}(v')$. Let v'' be the predecessor of v' . Notice that since there is a single path from u to v , we have that $\text{pre}(v) < \text{pre}(v')$ implies $\text{pre}(v) < \text{pre}(v'')$.⁴ Then we continue by induction until we get $\text{pre}(v) < \text{pre}(u)$, which is a contradiction. □

Claim 2. *Let $G = (V, E)$ be a DAG, and let $v, u \in V$ such $u \rightarrow v$ (There is a path from u to v in G). Then the call **DFS**(G) assigns $\text{post}(u) > \text{post}(v)$.*

Proof. Suppose, toward a contradiction, that $\text{post}(u) < \text{post}(v)$, and split into cases:

- Either $\text{pre}(v) < \text{pre}(u)$ thus, from the parenthesis theorem, $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$, and u is a descendant of v in π . Therefore, there is a path from v to u in G . Combining this with the given that there is a path from u to v , there is a cycle in G , which contradicts G being a DAG.
- Or, $\text{pre}(v) > \text{pre}(u)$, Thus, by Claim 1, we get that v is a descendant of u in π , and by combining the parenthesis theorem, we get the desired.

⁴That is, upon calling '**Explore**(G', v'')', the algorithm will first walk through paths that do not reach v , and then call '**Explore**(G', v')'.

□

Claim 3. Define the relation $v \sim u$ to hold if u and v are on the same strong connected components. Then:

1. $\sim$ is an equivalence relation.
2. Define the graph $G' = (V', E')$ such that V' are the equivalence class respecting to $\sim$ and $\{u, v\} \in E'$ if there is a directed path in G from a vertex in u into a vertex in v . Then G' is a DAG.

Claim 4. Let u the vertex with the minimal post value, then **Explore** (G, u) colorize only the vertices on u 's strong connected components.